

中国矿业大学计算机学院实验报告

课程名称	并行算法与 GPU 编程	实验名称	实验一 CPU 多核编程		
班级	大数据一班	姓名	王志豪	学号	08232337
实验日期	2026. 4. 25				
实验报告要求：1. 实验目的 2. 实验内容（题目描述，算法描述，运行截图，调试情况） 3. 实验体会					

一、实验目的

了解掌握多核并行程序设计，理解 CPU 多核并发编程模式。进程与线程概念，线程间通信（同步与互斥），包括：互锁函数、临界区（临界段），生产者与消费者同步等算法。

二、实验内容

2.1 题目描述

基于多核 CPU 平台，使用 POSIX 线程（pthread）库实现并行化的图像卷积算法，通过边缘检测任务掌握并行计算的基本方法与性能分析技术。

- 实现串行版本的图像卷积（边缘检测）算法，作为性能基准；
- 基于 pthread 多线程，将卷积计算任务并行化，利用多核 CPU 加速计算；
- 对比串行与并行的执行时间，计算加速比，分析并行性能；
- 验证并行计算结果的正确性（与串行结果完全一致）。

本次实验选择 3×3 拉普拉斯边缘检测算子作为卷积核，对标准 Lena 图像进行边缘提取，实现并行加速。

2.2 算法描述

本实验采用 3×3 拉普拉斯算子进行图像边缘检测。设输入图像为 $I(x,y)$ ，卷积核为

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

对于输出图像中位置为 (x,y) 的像素，其卷积结果可表示为

$$O(x,y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K(i,j) \cdot I(x+i,y+j)$$

该算子通过计算中心像素与其邻域像素之间的差异，突出图像中的灰度变化区域，因此能够实现边缘提取。

对于 8bit 灰度图像，每个像素只有一个通道，因此直接对单通道图像执行上述卷积即可。对于 24bit 彩色图像，设三个通道分别为 $B(x,y)$ 、 $G(x,y)$ 和 $R(x,y)$ ，则分别对三个通道独立执行卷积：

$$O_B(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K(i, j) \cdot B(x + i, y + j)$$

$$O_G(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K(i, j) \cdot G(x + i, y + j)$$

$$O_R(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K(i, j) \cdot R(x + i, y + j)$$

最终将 $O_B(x, y)$ 、 $O_G(x, y)$ 、 $O_R(x, y)$ 组合成输出彩色像素。

由于图像边界处邻域不完整，为保证每个像素都能进行 3×3 卷积，实验在图像四周补充 1 个像素宽度的 padding。若原图尺寸为 $M \times N$ ，则填充后图像尺寸为

$$(M + 2) \times (N + 2)$$

在边界补零后，卷积公式即可对整幅图像统一处理。

串行版本按像素顺序逐行遍历整幅图像，计算公式为

$$O(x, y) = f(I(x, y))$$

其中 $f(\cdot)$ 表示一次 3×3 拉普拉斯卷积运算。并行版本则将图像按行划分为若干子区域，设线程数为 T ，每个线程负责一部分连续行区间。若第 t 个线程负责的行范围为 $[y_t^{start}, y_t^{end})$ ，则其计算任务为

$$O(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 K(i, j) \cdot I(x + i, y + j), y \in [y_t^{start}, y_t^{end})$$

由于各线程处理的输出区域互不重叠，因此不需要额外加锁。最终将所有线程的结果合并，即得到并行卷积输出。

为了验证正确性，实验对串行结果与并行结果进行逐字节比较。若两者完全一致，则说明并行实现与串行实现等价。性能评价则通过测量串行时间 T_s 和并行时间 T_p ，并计算加速比

$$S = \frac{T_s}{T_p}$$

串行和并行的卷积操作本质上都是使用的相同的操作逻辑，所以实验中的卷积操作是串行和并行共用的，如下所示：

```

static void convolve_rows(const uint8_t *padded,
                          uint8_t *output,
                          int width,
                          int height,
                          int padded_width,
                          int row_begin,
                          int row_end)
{
    int y;
    int x;
    int channel;
    int ky;
    int kx;
    (void)height;
    for (y = row_begin; y < row_end; ++y)
    {
        for (x = 0; x < width; ++x)
        {
            for (channel = 0; channel < IMAGE_CHANNELS; ++channel)
            {
                int sum = 0;
                for (ky = 0; ky < 3; ++ky)
                {
                    const uint8_t *src_row = padded + ((size_t)(y + ky) * (size_t)padded_width + (size_t)x) * IMAGE_CHANNELS;
                    for (kx = 0; kx < 3; ++kx)
                    {
                        sum += kLaplacianKernel[ky][kx] * src_row[kx * IMAGE_CHANNELS + channel];
                    }
                }
                if (sum < 0)
                {
                    sum = -sum;
                }
                output[((size_t)y * (size_t)width + (size_t)x) * IMAGE_CHANNELS + channel] = clamp_byte(sum);
            }
        }
    }
}

```

对于彩色和灰度图像来说，灰度图像由于它是单通道的图像，因此，每一个像素点由一个二维数组中的一个表示，所以卷积操作是对一组矩阵的操作；而彩色图像由于 BMP 格式的特殊性，每三字节组成一个像素值，因此卷积核在处理的时候是要分三组（RGB）分别计算出结果并拼接在一起的。

以灰度图像为例，实验的步骤大致如下：

1. 先获取线程数 `thread_count`，如果没有传入，就默认取 CPU 逻辑核心数。
2. 把图像高度 `height` 平均分配给多个线程。
3. 每个线程拿到自己的行区间 `[row_begin, row_end)`。
4. 线程只对这段行范围做卷积，不访问别的线程负责的输出区域。
5. 所有线程执行完后，主线程汇总结果并继续后续的结果比较和写文件。

具体的多线程创建过程如下所示：

```

base_rows = height / thread_count;
remainder = height % thread_count;
row_begin = 0;
for (i = 0; i < thread_count; ++i)
{
    rows = base_rows + (i < remainder ? 1 : 0);
    args[i].padded = padded;
    args[i].output = output;
    args[i].width = width;
    args[i].height = height;
    args[i].padded_width = padded_width;
    args[i].row_begin = row_begin;
    args[i].row_end = row_begin + rows;

    if (pthread_create(&threads[i], NULL, worker_main, &args[i]) != 0)
    {
        fprintf(stderr, "Failed to create pthread worker %d\n", i);
        failed = 1;
        thread_count = i;
        goto join_and_cleanup;
    }

    row_begin += rows;
}

```

2.3 运行截图

对于灰度图像：

```

Input: lena_gray.bmp
Image: 512x512, 1 channel(s)
Threads: 20
Serial time: 11.483 ms
Parallel time: 4.287 ms
Speedup: 2.679x
Verification: serial and parallel outputs are identical.

```



可以由输出结果来看，我们的开启了 20 个独立的线程，最终加速比为 2.679，并行计算的速度显著快于串行计算的速度，并且 lena 图像的边缘特征提取也效果良好。

对于彩色图像：

```
Input: lena.bmp
Image: 512x512, 3 channel(s)
Threads: 20
Serial time: 28.253 ms
Parallel time: 8.011 ms
Speedup: 3.527x
Verification: serial and parallel outputs are identical.
```



可以从彩色图像的处理结果得出，彩色图像因为其多通道的特性，数据量明显大于灰度图像，串行和并行的计算时间也显著慢于灰度图像，相同的，并行计算要显著快于串行计算，并且，由加速比的指标对比来看，彩色图像的加速比高于灰度图像的加速比，说明对于大数据量来说，并行计算的加速效果要更显著。

2.4 调试情况

A. Padding 问题：

最初，我直接将卷积核与原始图像进行计算，最终 512×512 的图像会缩水成 510×510 的图像，因为卷积计算的特点，像素值会丢失，所以需要添加 padding。

在卷积计算中，为避免边界像素越界访问，程序对原图四周补充了 1 个像素宽度的零填充。调试时重点验证了 padding 后图像尺寸是否正确，以及卷积时邻域索引是否越界。通过对边界和非边界像素进行检查，确认 3×3 拉普拉斯卷积能够覆盖整幅图像。

B. 并行计算问题：

在并行实现中，程序使用 pthread 将图像按行划分为多个子区域，每个线程负责一部分连续行的卷积计算。调试时重点检查了线程分配是否均匀、线程处理范围是否重叠或遗漏，以及线程结束后结果是否完整。最终确认各线程只访问自己的输出区域，不存在数据竞争问题。

三、实验体会

A. CPU 多核并发编程模式：

这次实验让我对多核并行程序设计有了比较直接的认识，尤其是“并行不是把代码简单复制成多个线程”这一点印象很深。最开始我以为只要创建多个线程一起跑就能明显加速，但真正做下来以后才发现，线程怎么切分任务、怎么避免重复计算、怎么保证结果正确，才是并行程序设计的关键。像这次的图像卷积实验，我把整幅图按行划分给多个线程处理，每个线程只负责自己那部分输出区域，这样既减少了同步开销，也避免了线程之间的数据冲突。这个过程让我理解了多核 CPU 并发编程的基本模式：把独立、可并行的任务拆出来，让多个核心同时工作，同时尽量减少共享资源带来的竞争。

B. 对于进程与线程的理解：

通过这次实验，我也更清楚地分辨了进程和线程的区别。进程之间资源相对独立，切换开销较大；线程则共享同一进程的地址空间，切换更轻量，适合这种“同一份输入数据、多个计算单元同时处理”的场景。图像卷积实验特别适合用线程来做，因为所有线程都只读同一张 padded 图像，写入时分别写到不同的输出行，因此天然适合并行化。这让我体会到，线程并不是越多越好，而是要根据任务特点选择合适的并行粒度。对于这次 512×512 的图像，如果把线程切得太细，线程创建和调度开销反而会抵消并行收益；而按行划分则比较稳定，也容易验证正确性。

C. 对于同步与互斥的理解：

在同步与互斥方面，这次实验也让我对“什么时候需要锁、什么时候不需要锁”有了更清晰的理解。由于我采用的是按行分块的方式，每个线程只处理自己的输出区间，没有两个线程同时写同一块内存，所以实际上不需要复杂的互斥机制。这说明并行程序设计的一个重要原则是：尽量通过任务划分避免共享写入，而不是事后依赖锁去修补。如果采用更复杂的切块方式，或者多个线程需要访问同一份可变数据，那就必须考虑临界区、互斥锁、信号量等同步手段，否则很容易产生数据竞争。也正因为如此，我对“临界区”这个概念有了更实际的理解：它不是抽象名词，而是程序中那一段必须被保护、不能被多个线程同时进入的代码区域。

D. 对于生产者消费者的理解：

另外，这次实验让我对生产者与消费者模型也有了更直观的认识。虽然这次图像卷积程序没有显式使用生产者-消费者队列，但从思想上看，卷积任务的分发和执行过程其实和这种模式很接近：主线程像“生产者”一样负责准备输入、划分任务、发放工作；子线程像“消费者”一样取走自己的任务并执行计算。这个过程让我意识到，很多并行程序都可以抽象成任务生成、任务分配、任务执行、结果汇总这样的结构。以后如果处理更复杂的并发程序，比如一边读数据、一边处理、一边输出，就很可能真的需要用到生产者-消费者同步机制。

从结果上看，这次实验不仅实现了串行和并行两种版本，还验证了并行结果与串行结果完全一致，这一点很重要。因为并行化的目标不是改变算法本身，而是在不改变结果的前提下提高执行效率。通过比对串行和并行输出，我确认了多线程划分没有引入逻辑错误，也进一步体会到并行程序调试的难点：速度问题还比较容易观察，但正确性问题往往更隐蔽，需要通过逐字节比较、边界测试和不同线程数测试来验证。

总的来说，这次实验让我真正把“多核并发”从概念变成了可以动手实现的程序结构。我不仅理解了线程并行的基本设计方法，也更清楚了同步、互斥、临界区这些并发控制手段为什么存在、在什么场景下需要使用。对我来说，最有收获的是建立了这样一种思维：并行程序设计的核心不是单纯追求线程数量，而是在正确性、负载均衡和同步开销之间找到合适的平衡。这个认识对以后继续学习并行计算、操作系统和高性能程序设计都会很有帮助。

